NAME:	R.Hemalatha
ID:	2400030211
SECTION:	S-59
SUBJECT:	FULL STACK APPLICATION DEVELOPMENT

#SKILL - 8 Spring Boot – JPQL & Query Methods Module

Tasks:

1. Create a Product entity with the following fields:

a. id

b. name

c. category

d. price

<u>Output:-</u>

LibraryCont... | LibraryAppAp... | Course.java | CourseServic... | Product.java

```java
package com.klu.skillab8.entity;

import jakarta.persistence.*;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String category;
    private double price;

    public Product() {}

    public Product(String name, String category, double price) {
        this.name = name;
        this.category = category;
        this.price = price;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getCategory() {
        return category;
    }

    public double getPrice() {
        return price;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setCategory(String category) {
        this.category = category;
    }
```

2. Create a ProductRepository and add derived query methods:
a. findByCategory(String category)
b. findByPriceBetween(double min, double max)

Output:-

LibraryCont... | LibraryAppAp... | Course.java | Product.java | ProductRepos...

```java
package com.klu.skillab8.repository;

import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {

    // Derived Queries
    Show AOT-generated Implementation, Query, etc...
    List<Product> findByCategory(String category);

    Show AOT-generated Implementation, Query, etc...
    List<Product> findByPriceBetween(double min, double max);

    // JPQL Queries

    // a. Sorting products by price
    Show AOT-generated Implementation, Query, etc...
    @Query("SELECT p FROM Product p ORDER BY p.price ASC")
    List<Product> sortProductsByPrice();

    // b. Fetch products above a price value
    Show AOT-generated Implementation, Query, etc...
    @Query("SELECT p FROM Product p WHERE p.price > :price")
    List<Product> findExpensiveProducts(double price);

    // c. Fetch products by category
    Show AOT-generated Implementation, Query, etc...
    @Query("SELECT p FROM Product p WHERE p.category = :category")
    List<Product> findProductsByCategory(String category);
}
```

3. Write JPQL queries using @Query for:
a. Sorting products by price
b. Fetching products above a price value
c. Fetching products by category **Output:-**

```java
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Derived Queries
    Show AOT-generated Implementation, Query, etc...
    List<Product> findByCategory(String category);

    Show AOT-generated Implementation, Query, etc...
    List<Product> findByPriceBetween(double min, double max);

    // JPQL Queries

    // a. Sorting products by price
    Show AOT-generated Implementation, Query, etc...
    @Query("SELECT p FROM Product p ORDER BY p.price ASC")
    List<Product> sortProductsByPrice();

    // b. Fetch products above a price value
    Show AOT-generated Implementation, Query, etc...
    @Query("SELECT p FROM Product p WHERE p.price > :price")
    List<Product> findExpensiveProducts(double price);

    // c. Fetch products by category
    Show AOT-generated Implementation, Query, etc...
    @Query("SELECT p FROM Product p WHERE p.category = :category")
    List<Product> findProductsByCategory(String category);
}
```

4. Create REST controller endpoints to test query methods:
a. /products/category/{category}
b. /products/filter?min=&max=
c. /products/sorted d. /products/expensive/{price}

Output:-

```java
import org.springframework.beans.factory.annotation.Autowired;

@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    ProductRepository productRepository;

    // a. /products/category/{category}
    @GetMapping("/category/{category}")
    public List<Product> getByCategory(@PathVariable String category) {
        return productRepository.findByCategory(category);
    }

    // b. /products/filter?min=&max=
    @GetMapping("/filter")
    public List<Product> filterByPrice(@RequestParam double min,
                                       @RequestParam double max) {
        return productRepository.findByPriceBetween(min, max);
    }

    // c. /products/sorted
    @GetMapping("/sorted")
    public List<Product> getSortedProducts() {
        return productRepository.sortProductsByPrice();
    }

    // d. /products/expensive/{price}
    @GetMapping("/expensive/{price}")
    public List<Product> getExpensiveProducts(@PathVariable double price) {
        return productRepository.findExpensiveProducts(price);
    }
}
```

5. Insert sample product records and test using Postman.

Output:-

GET http://localhost:8080/products/category/Electronics

Docs Params Auth Headers (8) Body Scripts Tests Set

Query Params

Key	Value
Key	Value

Body 200 O

{} JSON Preview Visualize


```
1  [
2      {
3          "name": "Laptop",
4          "category": "Electronics",
5          "price": 75000.0,
6          "id": 1
7      },
8      {
9          "name": "Mobile",
10         "category": "Electronics",
11         "price": 30000.0,
12         "id": 2
13     },
14     {
15         "name": "Headphones",
16         "category": "Electronics",
17         "price": 2000.0,
18         "id": 7
```

GET http://localhost:8080/products/filter?min=1000&max=5000

Docs | Params | Auth | Headers (8) | Body | Scripts | Tests | Settings

Query Params

Key	Value	Descript
min	1000	
max	5000	
Key	Value	Descript

Body 200 OK 72

JSON | Preview | Visualize


```
[
    {
        "name": "Shoes",
        "category": "Fashion",
        "price": 2500.0,
        "id": 3
    },
    {
        "name": "T-Shirt",
        "category": "Fashion",
        "price": 1200.0,
        "id": 4
    },
    {
        "name": "Headphones",
        "category": "Electronics",
        "price": 2000.0,
        "id": 7
```


GET http://localhost:8080/products/sorted

Docs | Params | Auth | Headers (8) | Body | Scripts | Tests | S

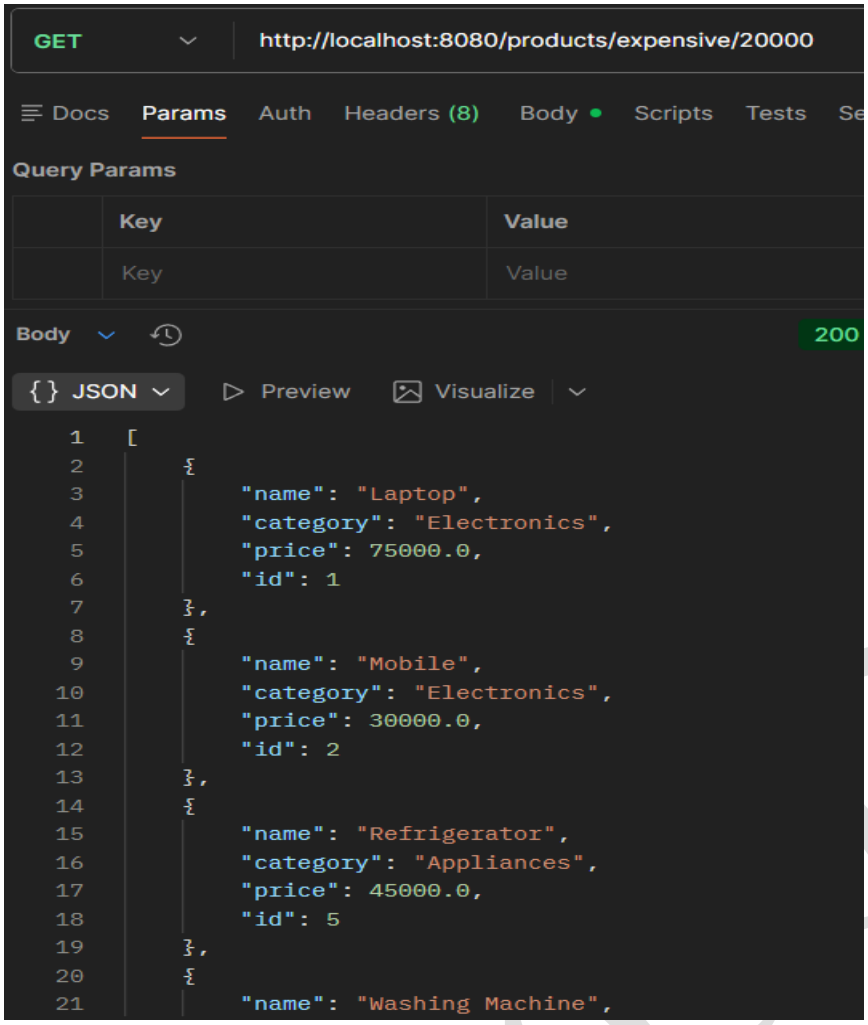
Query Params

Key	Value
Key	Value

Body 200

JSON | Preview | Visualize


```
[
    {
        "name": "T-Shirt",
        "category": "Fashion",
        "price": 1200.0,
        "id": 4
    },
    {
        "name": "Headphones",
        "category": "Electronics",
        "price": 2000.0,
        "id": 7
    },
    {
        "name": "Shoes",
        "category": "Fashion",
        "price": 2500.0,
        "id": 3
    },
    {
        "name": "Mobile",
```



7. Push the Spring Boot project to GitHub.

GITHUB REPO LINK :- https://github.com/hemalatharachakonda/2400030211-Skill_experiments.git

VIVA:

1. What is JPQL and how is it different from SQL?
Answer:
JPQL (Java Persistence Query Language) is used to query entities in JPA using entity class names and attributes. SQL queries database tables directly, while JPQL works with Java objects.

2. What are derived query methods in Spring Data JPA?
Answer:
Derived query methods are query methods automatically generated by Spring Data JPA based on method names like findByName() or findByEmail().

3. Why is method naming important in Spring Data JPA?
Answer:
Method naming is important because Spring Data JPA automatically creates queries by analyzing the method name. The name must follow specific conventions like findBy, deleteBy, or countBy.

4. Can JPQL return custom objects in a query?
Answer:
Yes, JPQL can return custom objects using constructor expressions such as SELECT new com.example.DTO(...).

2400030211

5. When should we choose derived queries instead of JPQL?
Answer:
Derived queries should be used for simple queries because they are easy to write and reduce boilerplate code compared to writing JPQL manually.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation: